EXPERIMENT 10

Creating Simple API Endpoints — GET & POST Requests in Flask

Aim

To build RESTful API endpoints in Flask that handle GET and POST requests and exchange data in JSON format.

Theory

A REST API exposes resources over HTTP using standard methods: GET (read), POST (create), PUT/PATCH (update) and DELETE (remove). APIs communicate with structured data — usually JSON — instead of HTML, so they can be consumed by front-ends, mobile apps or other services.

In Flask, the HTTP methods a route accepts are declared via 'methods=[...]'. Incoming JSON is read with request.get_json(); responses are produced with jsonify(), which serialises a Python dict to JSON and sets the correct Content-Type header.

Well-designed endpoints return appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found) so clients can react programmatically. This request/response contract is the backbone of serving ML predictions as a service.

Software / Tools Required

Flask, curl or Postman (for testing), Python requests (optional).

Procedure (Step-by-Step)

Step 1. Create an in-memory data store and a GET endpoint returning JSON.

```
# api.py
from flask import Flask, request, jsonify
app = Flask(__name__)
items = [{"id": 1, "name": "GPU"}, {"id": 2, "name": "TPU"}]

@app.route("/items", methods=["GET"])
def get_items():
    return jsonify(items), 200
```

Step 2. Add a GET endpoint that fetches a single item by id, with a 404.

```
@app.route("/items/<int:id>", methods=["GET"])
def get_item(id):
    item = next((i for i in items if i["id"] == id), None)
    return (jsonify(item), 200) if item else (jsonify({"error":"not found"}),
404)
```

Step 3. Add a POST endpoint that creates a new item from JSON input.

```
@app.route("/items", methods=["POST"])
def add_item():
    data = request.get_json()
    if not data or "name" not in data:
        return jsonify({"error": "name required"}), 400
    new = {"id": len(items)+1, "name": data["name"]}
    items.append(new)
    return jsonify(new), 201
```

Step 4. Run the API server.

```
python api.py
```

Step 5. Test the GET endpoint with curl.

```
curl http://127.0.0.1:5000/items
```

Step 6. Test the POST endpoint by sending JSON.

```
curl -X POST http://127.0.0.1:5000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "FPGA"}'
```

Step 7. Verify the new item was created.

```
curl http://127.0.0.1:5000/items/3
```

Step 8. Test error handling with a bad request.

```
curl -X POST http://127.0.0.1:5000/items \
  -H "Content-Type: application/json" -d '{}'   # returns 400
```

Step 9. Consume the API programmatically from Python.

```
import requests
r = requests.post("http://127.0.0.1:5000/items", json={"name":"ASIC"})
print(r.status_code, r.json())
```

Step 10. Add a health-check endpoint (common in production).

```
@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200
```

Expected Output

GET /items returns the JSON array with status 200; POST /items with a valid body creates a new item and returns it with status 201; an empty POST body returns a 400 error — confirming correct method handling and status codes.

Conclusion

GET and POST REST endpoints exchanging JSON with proper status codes were implemented in Flask, providing the API pattern used later to serve ML model predictions.

Viva Questions

1. What is the difference between GET and POST?
2. What does jsonify() do that returning a dict directly does not (historically)?
3. Name the HTTP status codes for created, bad request and not found.
4. How do you read a JSON body from a Flask request?
5. What makes an API 'RESTful'?